

Harry Myers 5/12/26

CS m146 HW#2

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

1. $J_{hinge}(w,b) = \sum_{n=1}^N \max(0, 1 - y_n(w^T x_n + b)) + \frac{\lambda}{2} \|w\|_2^2$ $y_n \in \{+1, -1\}$, $\lambda > 0$

logistic loss $J_{log}(w,b) = \sum_{n=1}^N \log(1 + e^{-y_n(w^T x_n + b)}) + \frac{\lambda}{2} \|w\|_2^2$, $m_n = y_n(w^T x_n + b) = \text{margin}$

a. I. $J_{log}(w,b) = \log(1 + e^{-y(w^T x + b)})$ let $m = y(w^T x + b) = \text{margin}$ $\frac{\partial}{\partial m} \log(1 + e^{-m}) = \frac{-e^{-m}}{1 + e^{-m}} = -\frac{e^{-m}}{1 + e^{-m}} = -\frac{1}{1 + e^m}$

a. II. $\frac{\partial}{\partial b} J_{log}(w,b) = \frac{-e^{-m}}{1 + e^{-m}} \cdot y = \frac{-e^{-m}}{1 + e^{-m}} = -\frac{1}{1 + e^m} = -\sigma(-m) = -\sigma(m)$

b. $\frac{\partial}{\partial w} J_{log} = \sum_{n=1}^N \frac{-y_n x_n}{1 + e^{m_n}} + w$ $\frac{\partial}{\partial w} \frac{\lambda}{2} \|w\|_2^2 = \lambda w$ $\frac{\partial}{\partial b} J_{log} = \sum_{n=1}^N \frac{-y_n}{1 + e^{m_n}} = -\sum_{n=1}^N \sigma(m_n) y_n$

c. $\nabla_w J_{log} = \frac{-e^{-m_n}}{1 + e^{-m_n}} y_n x_n$ $m_n = y_n(w^T x_n + b)$ $\frac{e^{-m_n}}{1 + e^{-m_n}} = \frac{1}{1 + e^{m_n}}$ is between 0 & 1

if $m \gg 0$ then $m_n \rightarrow 0$, if $m \approx 0$ $m_n \rightarrow \frac{1}{2}$, if $m \ll 0$, $m_n \rightarrow 1$ So hinge loss bounded

I. A correctly classified example far away from decision boundary doesn't influence the weights

II. $m \approx 0$ - close to boundary so margin of $1/2$ unsure so pull harder on the boundary

III. $m \ll 0$ confidently miss classified $\rightarrow$ so badly classified contributes the strongest m_n is bounded at 1 no matter how wrong

d. $J_{hinge}(w,b) = \max(0, 1 - y(w^T x + b))$ max & margin; logistic $J_{log} = \frac{1}{1 + e^m}$

I. $m \geq 1: 0 \rightarrow \nabla_w \begin{cases} 0 & m \geq 1 \\ -y x & m < 1 \end{cases}$ Hinge loss contributes to zero gradient when $m \geq 1$

II. Note it can not but the denominator can get very big

III. correctly classified with large positive margin $\Rightarrow$ that is log loss is in hinge loss exactly 0.

E. $m \ll 0$ so confidently missclassified & magnitude is bounded at 1 for log.

For hinge if $m \ll 0$ then $J_{hinge} = -y x$ so it is not dependent on m & no matter how missclassified

you are hinge loss treats the same, $m = 9 \rightarrow -1000$ doesn't matter they both reach the same

because $-y x = x$ is 1 & $-y = -1$ so both approach 1.

F. $h(w) = \text{sign}(w^T x + b)$ replace hinge with log choose boundary of training objective?

The boundary is defined on where it switches sign: $w^T x + b = 0 \rightarrow$ is dependent on equation not how we bound

But they do produce different values of w & b because hinge only when $m \leq 1$ only listens to support vectors while log listens to every vector.

$$k(x, z) = \phi(x)^T \phi(z)$$

2. Gaussian RBF kernel $= \exp(-\|x - z\|^2)$, $x, z \in \mathbb{R}^D$ a. $\|x - z\|^2 = (x - z)^T (x - z) = x^T x - 2x^T z + z^T z$

$k(x, z) = \exp(-(x^T x - 2x^T z + z^T z)) = \exp(-x^T x + 2x^T z - z^T z)$ $\leftarrow$ only dot products

b. $d_k(x, z) = \|\phi(x) - \phi(z)\|^2 = (\phi(x) - \phi(z))^T (\phi(x) - \phi(z)) = \phi(x)^T \phi(x) - 2\phi(x)^T \phi(z) + \phi(z)^T \phi(z)$

$= k(x, x) - 2k(x, z) + k(z, z) = \text{kernel trick}$

II. $k(x, x) = \exp(-\|x - x\|^2) = \exp(0) = 1$ $\leftarrow$ every point of $\phi(x)$ lies on unit sphere so $\|x\|^2 = 1$

III. $d_k(x, z) = 2 - 2k(x, z)$

HW #2 P. 2

- ▷ Z.C. $d_k(x, z) = z - z k(x, z)$ where $k(x, z) = \phi(x)^T \phi(z) = \exp(-\|x - z\|^2)$ $x, z \in \mathbb{R}^d$
 ▷ I. $\|x - z\| \rightarrow 0$ then $k(x, z) = 1$ and $d_k(x, z) = 0$ points that are close in input are close in feature
 ▷ II. $\|x - z\| \rightarrow \infty$ then $k(x, z) \rightarrow 0$ and $d_k(x, z) = z$ no matter how far apart x, z they're feature space dist is mapped at z .
 ▷ Since they all sit on unit sphere max dist is π but it's mapped to z bounded region $x, z \in \mathbb{R}^d$
 (Classifier of this form) $d. h(x) = \sum_{n=1}^N \alpha_n k(x_n, x)$ $h(x)$? when test point x is far from all training points $\{x_n\}_1^N$?
 $\|x - x_n\| \gg 0$ for every n so $k(x_n, x) \rightarrow 0$ function $h(x) = \sum_{n=1}^N \alpha_n k(x_n, x) \rightarrow 0$
 ▷ model becomes uncertain and says I don't know for this classifier ✓
 ▷ E. Everything we need was computed using $k(x, z)$ & never $\phi(x)$ directly
 ex: Feature Space distances $k(x, x) + k(z, z) - 2k(x, z)$, Predictions $\sum \alpha_n k(x_n, x)$, and Training $k(x, x_i) = k(x_i, x)$
 ▷ RBFs kernel feature space $\in \mathbb{R}^\infty$ Taylor expansion of $e^{-\|x - z\|^2}$ but kernel is 64/128 (old) to 64/128
 ▷ 3. $k(x, z) = \|x - z\|^2$ $x, z \in \mathbb{R}^d$ α_n assume $k(x, z) = \phi(x)^T \phi(z)$
 $d_k(x, z) = \|\phi(x) - \phi(z)\|^2$ I. $= (\phi(x) - \phi(z))^T (\phi(x) - \phi(z)) = \phi(x)^T \phi(x) - 2\phi(x)^T \phi(z) + \phi(z)^T \phi(z)$
 $= k(x, x) + k(z, z) - 2k(x, z)$ II. $k(x, x) = 0 \leftarrow$ every point maps to zero vector?
 ▷ III. $d_k(x, z) = -2k(x, z) = -2\|x - z\|^2$
 b. d_k can not be a valid squared dist because it's always neg & squared dist should be +, Also kernel measures similarity so $k(x, x)$ should be pretty similar and not 0.
 ▷ C. $K = \begin{bmatrix} k(x_1, x_1) & k(x_1, x_2) \\ k(x_2, x_1) & k(x_2, x_2) \end{bmatrix}$ I. compute K , $x_1, x_2 \in \mathbb{R}^d$ $\delta = \|x_1 - x_2\|^2 > 0$
 then $k(x_1, x_1) = 0$ $k(x_1, x_2) = \|x_1 - x_2\|^2 = \delta = \begin{bmatrix} 0 & \delta \\ \delta & 0 \end{bmatrix}$
 ▷ II. $\begin{bmatrix} -1 & \delta \\ \delta & -1 \end{bmatrix}$ $\lambda^2 - \delta^2 = 0$ $\lambda = \pm \delta$ $\lambda = \pm \delta$
 ▷ III. Is K PSD? meaning all eigenvalues are ≥ 0 but this is not true $\lambda_2 = -\|x_1 - x_2\|^2 < 0$
 d. $k(x, z) = \|x - z\|^2$ is not valid kernel because
 1. $d_k(x, z) < 0$ This must be ≥ 0
 2. $\lambda = -\delta < 0$ This also must be ≥ 0 All eigenvalues of gram matrix must be ≥ 0 . & all gram matrixes must be PSD.
 3. Squared euclidean distance shows that kernel should be large when points are close because it measures similarity!

Problem 4: Kernelized SVM and Decision-Boundary Geometry

We explore how kernel choice reshapes the SVM decision boundary using two features from the Heart Disease UCI dataset:

- **x1** (`thalach`): maximum heart rate achieved (bpm)
- **x2** (`oldpeak`): ST depression induced by exercise (mm)
- **y** (`target`): 1 if heart disease, 0 otherwise

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import warnings
warnings.filterwarnings('ignore')

from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

plt.rcParams['figure.dpi'] = 100
plt.rcParams['font.size'] = 11
```

(a) Exploratory Setup

(a)(i) Load and Scatter

```
In [4]: df = pd.read_csv('HW_2_Datasets/HW_2_Datasets/HW_4/heart_disease.csv')
print(df.shape)
print(df.head())
print(df.describe())
print(f"\nClass balance: {df['target'].value_counts().to_dict()}")
```



```
(303, 3)
      thalach  oldpeak  target
0      150.0      2.3      0
1      108.0      1.5      1
2      129.0      2.6      1
3      187.0      3.5      0
4      172.0      1.4      0
```

	thalach	oldpeak	target
count	303.000000	303.000000	303.000000
mean	149.607261	1.039604	0.458746
std	22.875003	1.161075	0.499120
min	71.000000	0.000000	0.000000
25%	133.500000	0.000000	0.000000
50%	153.000000	0.800000	0.000000
75%	166.000000	1.600000	1.000000
max	202.000000	6.200000	1.000000

Class balance: {0: 164, 1: 139}

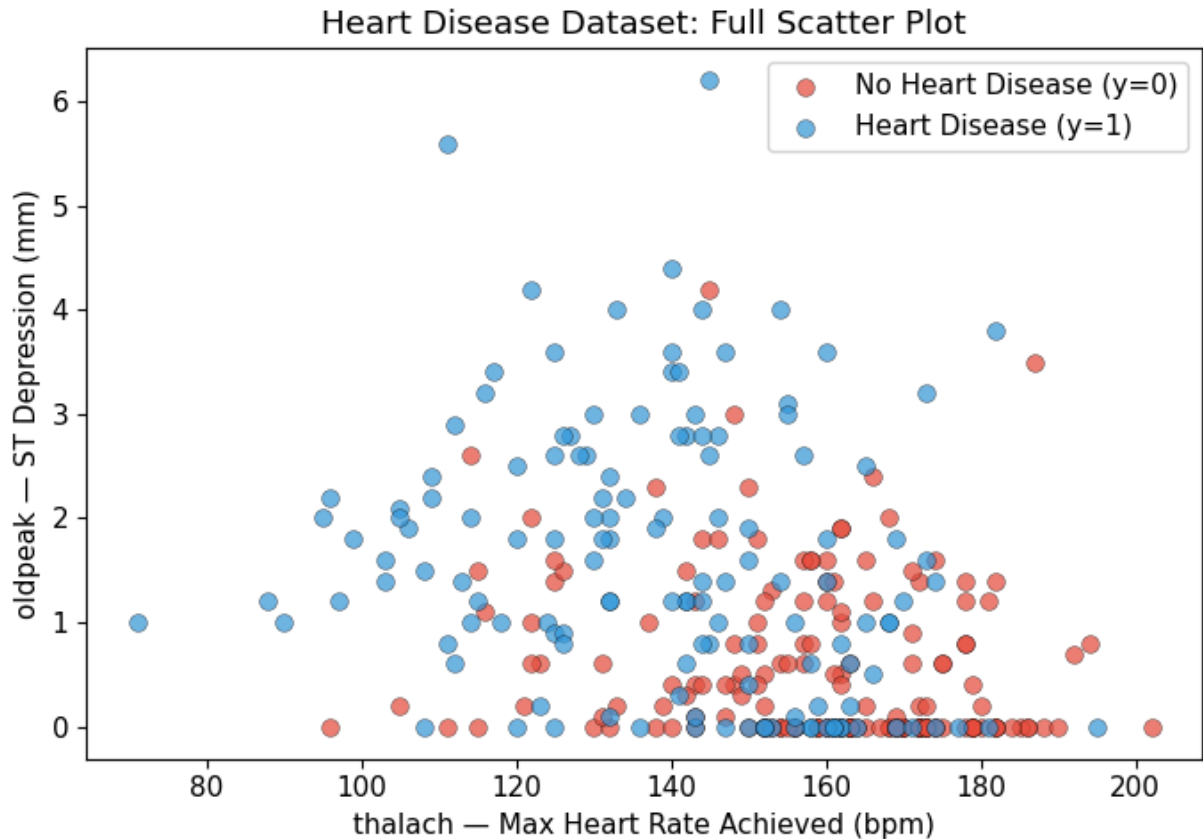
```
In [5]: fig, ax = plt.subplots(figsize=(7, 5))

colors = {0: '#e74c3c', 1: '#3498db'}
labels_map = {0: 'No Heart Disease (y=0)', 1: 'Heart Disease (y=1)'}

for label, group in df.groupby('target'):
    ax.scatter(group['thalach'], group['oldpeak'],
               c=colors[label], label=labels_map[label],
               alpha=0.7, edgecolors='k', linewidths=0.3, s=50)

ax.set_xlabel('thalach - Max Heart Rate Achieved (bpm)')
ax.set_ylabel('oldpeak - ST Depression (mm)')
ax.set_title('Heart Disease Dataset: Full Scatter Plot')
ax.legend()
plt.tight_layout()
plt.savefig('p4_a_scatter.png', bbox_inches='tight')
plt.show()

print("""
Visual inspection:
- The two classes are NOT linearly separable by eye.
- There is substantial overlap in the middle region (thalach ≈ 130-160 bpm, oldpeak ≈ 0.5-2.5 mm).
- Patients with heart disease tend toward higher heart rates and lower ST depression,
  while healthy patients cluster at lower heart rates and higher ST depression,
  but these trends overlap considerably.
- Because the data is not linearly separable, fancier (non-linear) kernels may help
  capture more complex decision boundaries.
""")
```



Visual inspection:

- The two classes are NOT linearly separable by eye.
- There is substantial overlap in the middle region (thalach $\approx$ 130–160 bpm, oldpeak $\approx$ 0–2 mm).
- Patients with heart disease tend toward higher heart rates and lower ST depression, while healthy patients cluster at lower heart rates and higher ST depression, but these trends overlap considerably.
- Because the data is not linearly separable, fancier (non-linear) kernels may help capture more complex decision boundaries.

(a)(ii) Train/Test Split and Standardization

```
In [6]: X = df[['thalach', 'oldpeak']].values
        y = df['target'].values

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )

        # Fit scaler on training set ONLY, then apply to both splits
        scaler = StandardScaler()
        X_train_std = scaler.fit_transform(X_train)
        X_test_std = scaler.transform(X_test)

        print(f"Training set: {X_train_std.shape[0]} samples")
        print(f"Test set: {X_test_std.shape[0]} samples")
        print(f"Train mean: {X_train_std.mean(axis=0)} (should be ~0)")
```

```
print(f"Train std:      {X_train_std.std(axis=0)}      (should be ~1)")

print("""
Why computing  $\mu$ ,  $\sigma$  on combined train+test data is incorrect (data leakage):
  If we standardize using the full dataset's statistics, information about the test
  set (its mean and variance) leaks into the training pipeline. This means the model
  has implicitly 'seen' the test set during preprocessing, invalidating the test-set
  accuracy as an honest estimate of generalization performance. The correct protocol
  is to fit  $\mu$  and  $\sigma$  exclusively on training data and apply those fixed values to
  transform the test set.
""")
```

```
Training set: 242 samples
Test set:     61 samples
Train mean:   [ 4.45236134e-16 -2.62875121e-16] (should be ~0)
Train std:    [1. 1.] (should be ~1)
```

Why computing μ , σ on combined train+test data is incorrect (data leakage):

If we standardize using the full dataset's statistics, information about the test set (its mean and variance) leaks into the training pipeline. This means the model has implicitly 'seen' the test set during preprocessing, invalidating the test-set accuracy as an honest estimate of generalization performance. The correct protocol is to fit μ and σ exclusively on training data and apply those fixed values to transform the test set.

(b) Linear SVM Baseline

Helper: Decision Boundary Plot Function

```
In [7]: def plot_decision_boundary(ax, clf, X_train_std, y_train, title,
                                     show_margin=False, show_sv=False,
                                     margin=0.5, resolution=300):

    """Plot a classifier's decision boundary on a 2D feature space."""
    x_min = X_train_std[:, 0].min() - margin
    x_max = X_train_std[:, 0].max() + margin
    y_min = X_train_std[:, 1].min() - margin
    y_max = X_train_std[:, 1].max() + margin

    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, resolution),
        np.linspace(y_min, y_max, resolution)
    )
    grid = np.c_[xx.ravel(), yy.ravel()]
    Z = clf.predict(grid).reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.2, cmap='bwr', levels=[-0.5, 0.5, 1.5])
    ax.contour(xx, yy, Z, levels=[0.5], colors='k', linewidths=1.5)

    # Margin Lines for Linear SVM
    if show_margin and hasattr(clf, 'coef_'):
        w = clf.coef_[0]
```

```

b = clf.intercept_[0]
x_vals = np.linspace(x_min, x_max, 200)
if abs(w[1]) > 1e-10:
    db = (-w[0] * x_vals - b) / w[1]
    marg_pos = (-w[0] * x_vals - b + 1) / w[1]
    marg_neg = (-w[0] * x_vals - b - 1) / w[1]
    ax.plot(x_vals, db, 'k-', lw=2, label='Decision boundary')
    ax.plot(x_vals, marg_pos, 'k--', lw=1, label='Margin ±1')
    ax.plot(x_vals, marg_neg, 'k--', lw=1)

# Training points
colors = {0: '#e74c3c', 1: '#3498db'}
for label in [0, 1]:
    mask = y_train == label
    ax.scatter(X_train_std[mask, 0], X_train_std[mask, 1],
               c=colors[label], alpha=0.6, edgecolors='k',
               linewidths=0.3, s=30, zorder=3)

# Support vectors
if show_sv and hasattr(clf, 'support_vectors_'):
    sv = clf.support_vectors_
    ax.scatter(sv[:, 0], sv[:, 1], s=120, facecolors='none',
               edgecolors='gold', linewidths=2, zorder=4,
               label='Support vectors')

ax.set_xlim(x_min, x_max)
ax.set_ylim(y_min, y_max)
ax.set_xlabel('thalach (standardized)')
ax.set_ylabel('oldpeak (standardized)')
ax.set_title(title)

```

(b)(i) Decision Boundary with Margin Lines and Support Vectors

```

In [8]: svc_linear = SVC(kernel='linear', C=1.0)
svc_linear.fit(X_train_std, y_train)

fig, ax = plt.subplots(figsize=(7, 5))
plot_decision_boundary(ax, svc_linear, X_train_std, y_train,
                       title='Linear SVM (C=1.0)\nDecision Boundary, Margin Lines,
                              show_margin=True, show_sv=True)

# Legend patches
red_patch = mpatches.Patch(color='#e74c3c', alpha=0.6, label='No Disease (y=0)')
blue_patch = mpatches.Patch(color='#3498db', alpha=0.6, label='Heart Disease (y=1)')
sv_handle = plt.Line2D([0], [0], marker='o', color='w',
                       markeredgecolor='gold', markerfacecolor='none',
                       markersize=10, markeredgewidth=2, label='Support Vectors')
db_handle = plt.Line2D([0], [0], color='k', lw=2, label='Decision Boundary')
mg_handle = plt.Line2D([0], [0], color='k', lw=1, ls='--', label='Margin ±1')

ax.legend(handles=[red_patch, blue_patch, sv_handle, db_handle, mg_handle],
          fontsize=9, loc='upper right')

print(f"Number of support vectors: {len(svc_linear.support_vectors_)}")

```

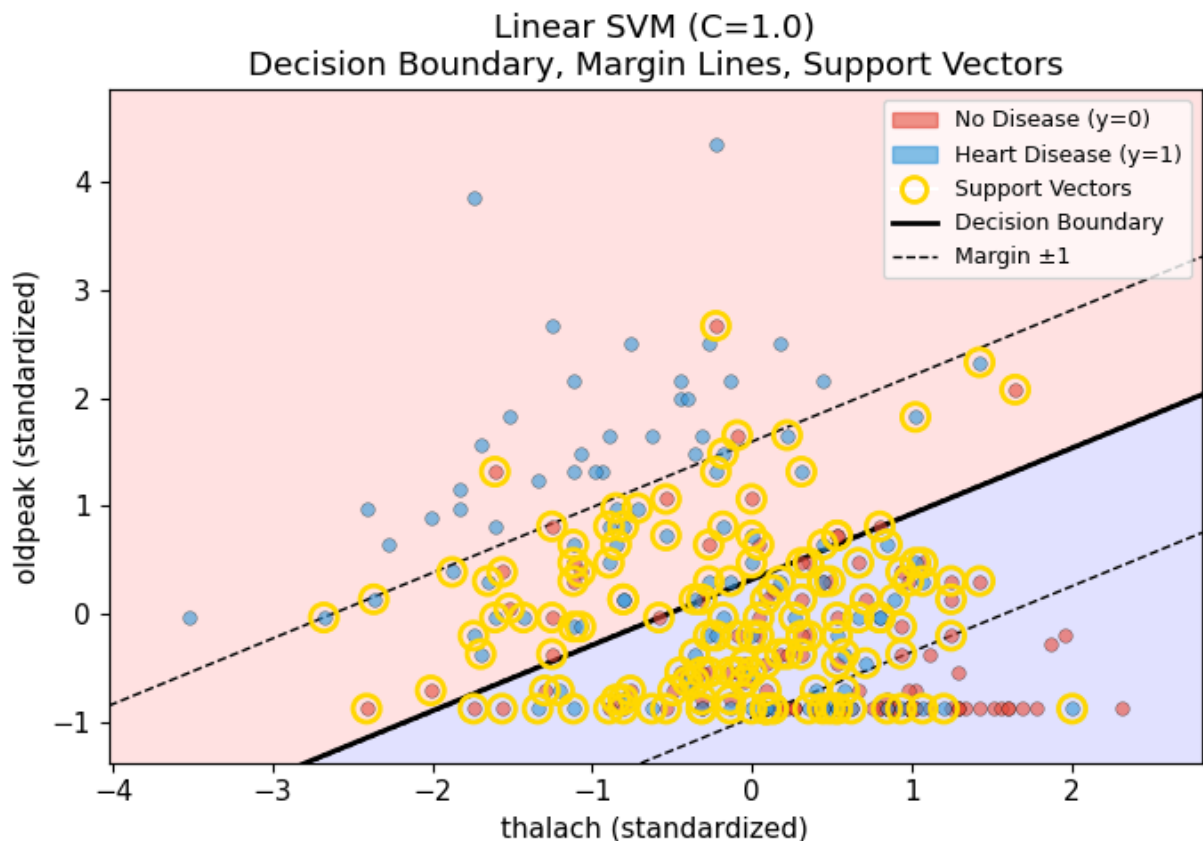
```
print(f" Class 0: {svc_linear.n_support_[0]}, Class 1: {svc_linear.n_support_[1]}")

plt.tight_layout()
plt.savefig('p4_b_linear_boundary.png', bbox_inches='tight')
plt.show()

print("""
Connection to theory (Problem 1):
The support vectors (gold circles) are precisely the training examples with
margin  $m_n = y_n(w^T x_n + b) \leq 1$ . These are the only examples contributing
non-zero subgradient to the hinge-loss objective. All other training points
(outside the margin band) have  $m_n > 1$  and contribute zero subgradient.
""")
```

Number of support vectors: 154

Class 0: 77, Class 1: 77



Connection to theory (Problem 1):

The support vectors (gold circles) are precisely the training examples with margin $m_n = y_n(w^T x_n + b) \leq 1$. These are the only examples contributing non-zero subgradient to the hinge-loss objective. All other training points (outside the margin band) have $m_n > 1$ and contribute zero subgradient.

(b)(ii) Training and Test Accuracy

```
In [9]: train_acc_linear = accuracy_score(y_train, svc_linear.predict(X_train_std))
test_acc_linear = accuracy_score(y_test, svc_linear.predict(X_test_std))

print(f"Linear SVM (C=1.0)")
```



```
print(f" Training accuracy: {train_acc_linear:.4f} ({train_acc_linear*100:.1f}%)")
print(f" Test accuracy:      {test_acc_linear:.4f} ({test_acc_linear*100:.1f}%)")
```

Linear SVM (C=1.0)

Training accuracy: 0.7231 (72.3%)
Test accuracy: 0.7705 (77.0%)

(c) Polynomial Kernels

(c)(i) Accuracy for Degrees {2, 3, 5}

```
In [10]: degrees = [2, 3, 5]
poly_svcs = {}
poly_results = {}

print("Polynomial Kernel SVM (C=1.0)")
print(f"{'Degree':<10} {'Train Acc':<14} {'Test Acc':<14}")
print("-" * 38)

for d in degrees:
    clf = SVC(kernel='poly', degree=d, C=1.0, coef0=1)
    clf.fit(X_train_std, y_train)
    tr = accuracy_score(y_train, clf.predict(X_train_std))
    te = accuracy_score(y_test, clf.predict(X_test_std))
    poly_svcs[d] = clf
    poly_results[d] = (tr, te)
    print(f"{'d':<10} {'tr':.4f} {'te':.4f}")
```

Polynomial Kernel SVM (C=1.0)

Degree	Train Acc	Test Acc
2	0.7190	0.7213
3	0.7273	0.7541
5	0.7355	0.7541

(c)(ii) Decision Boundary Grid (1×3)

```
In [11]: fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for ax, d in zip(axes, degrees):
    clf = poly_svcs[d]
    tr, te = poly_results[d]
    plot_decision_boundary(
        ax, clf, X_train_std, y_train,
        title=f'Polynomial Kernel (degree={d})\nTrain: {tr:.3f} | Test: {te:.3f}',
        show_margin=False, show_sv=False
    )

# Shared Legend
red_patch = mpatches.Patch(color='#e74c3c', alpha=0.6, label='No Disease (y=0)')
blue_patch = mpatches.Patch(color='#3498db', alpha=0.6, label='Heart Disease (y=1)')
axes[-1].legend(handles=[red_patch, blue_patch], fontsize=9, loc='upper right')
```

```
plt.suptitle('Polynomial SVM Decision Boundaries (C=1.0)', y=1.01, fontsize=13)
plt.tight_layout()
plt.savefig('p4_c_poly_boundaries.png', bbox_inches='tight')
plt.show()
```

```
print("""
```

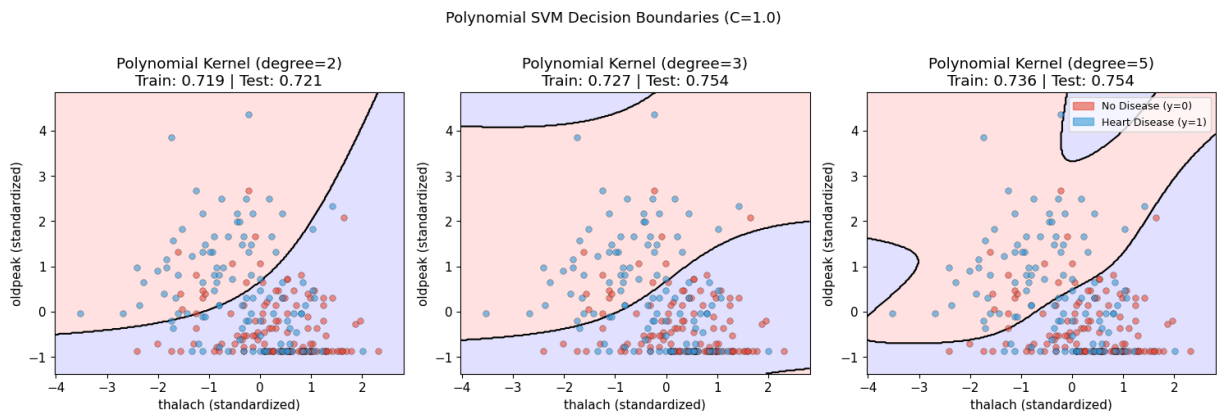
Observations on how the boundary changes with degree:

degree=2: The boundary is a smooth conic section (ellipse/hyperbola/parabola), gently curving to separate the two classes. It generalizes well.

degree=3: The boundary becomes more flexible with an S-shaped or wavy curve, fitting some of the intermediate overlap region more closely.

degree=5: Overfitting starts to become visually obvious. The boundary makes unphysical excursions—sharp hooks and isolated islands—to chase individual training points. The complex shape does not reflect the underlying data distribution and is likely to generalize poorly on unseen data.

```
""")
```



Observations on how the boundary changes with degree:

degree=2: The boundary is a smooth conic section (ellipse/hyperbola/parabola), gently curving to separate the two classes. It generalizes well.

degree=3: The boundary becomes more flexible with an S-shaped or wavy curve, fitting some of the intermediate overlap region more closely.

degree=5: Overfitting starts to become visually obvious. The boundary makes unphysical excursions—sharp hooks and isolated islands—to chase individual training points. The complex shape does not reflect the underlying data distribution and is likely to generalize poorly on unseen data.

(d) RBF (Gaussian) Kernel

(d)(i) Accuracy for $\gamma \in \{0.1, 1, 10, 100\}$

```
In [12]: gammas = [0.1, 1, 10, 100]
rbf_svcs = {}
rbf_results = {}

print("RBF Kernel SVM (C=1.0)")
print(f"{'gamma':<10} {'Train Acc':<14} {'Test Acc':<14}")
print("-" * 38)

for g in gammas:
    clf = SVC(kernel='rbf', gamma=g, C=1.0)
    clf.fit(X_train_std, y_train)
    tr = accuracy_score(y_train, clf.predict(X_train_std))
    te = accuracy_score(y_test, clf.predict(X_test_std))
    rbf_svcs[g] = clf
    rbf_results[g] = (tr, te)
    print(f"{'g':<10} {'tr':.4f} {'te':.4f}")
```

RBF Kernel SVM (C=1.0)		
gamma	Train Acc	Test Acc
0.1	0.7397	0.7541
1	0.7355	0.8197
10	0.8017	0.6885
100	0.9215	0.7213

(d)(ii) Decision Boundary Grid (2×2)

```
In [13]: fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes_flat = axes.ravel()

for ax, g in zip(axes_flat, gammas):
    clf = rbf_svcs[g]
    tr, te = rbf_results[g]
    plot_decision_boundary(
        ax, clf, X_train_std, y_train,
        title=f'RBF SVM (γ={g})\nTrain: {tr:.3f} | Test: {te:.3f}',
        show_margin=False, show_sv=False
    )

# Shared Legend
red_patch = mpatches.Patch(color='#e74c3c', alpha=0.6, label='No Disease (y=0)')
blue_patch = mpatches.Patch(color='#3498db', alpha=0.6, label='Heart Disease (y=1)')
axes_flat[-1].legend(handles=[red_patch, blue_patch], fontsize=9, loc='upper right')

plt.suptitle('RBF SVM Decision Boundaries (C=1.0)', y=1.01, fontsize=13)
plt.tight_layout()
plt.savefig('p4_d_rbf_boundaries.png', bbox_inches='tight')
plt.show()

best_g = max(rbf_results, key=lambda g: rbf_results[g][1])
overfit = max(rbf_results, key=lambda g: rbf_results[g][0] - rbf_results[g][1])

print(f"Best test accuracy: γ = {best_g} "
      f"(train={rbf_results[best_g][0]:.3f}, test={rbf_results[best_g][1]:.3f})")
print(f"Most overfit: γ = {overfit} ")
```



```
f"(train={rbf_results[overfit][0]:.3f}, test={rbf_results[overfit][1]:.3f})")
print("""
Connection to Problem 2(b)(iii):
The feature-space distance is  $d_k(x,z) = 2(1 - \exp(-\gamma||x-z||^2))$ .

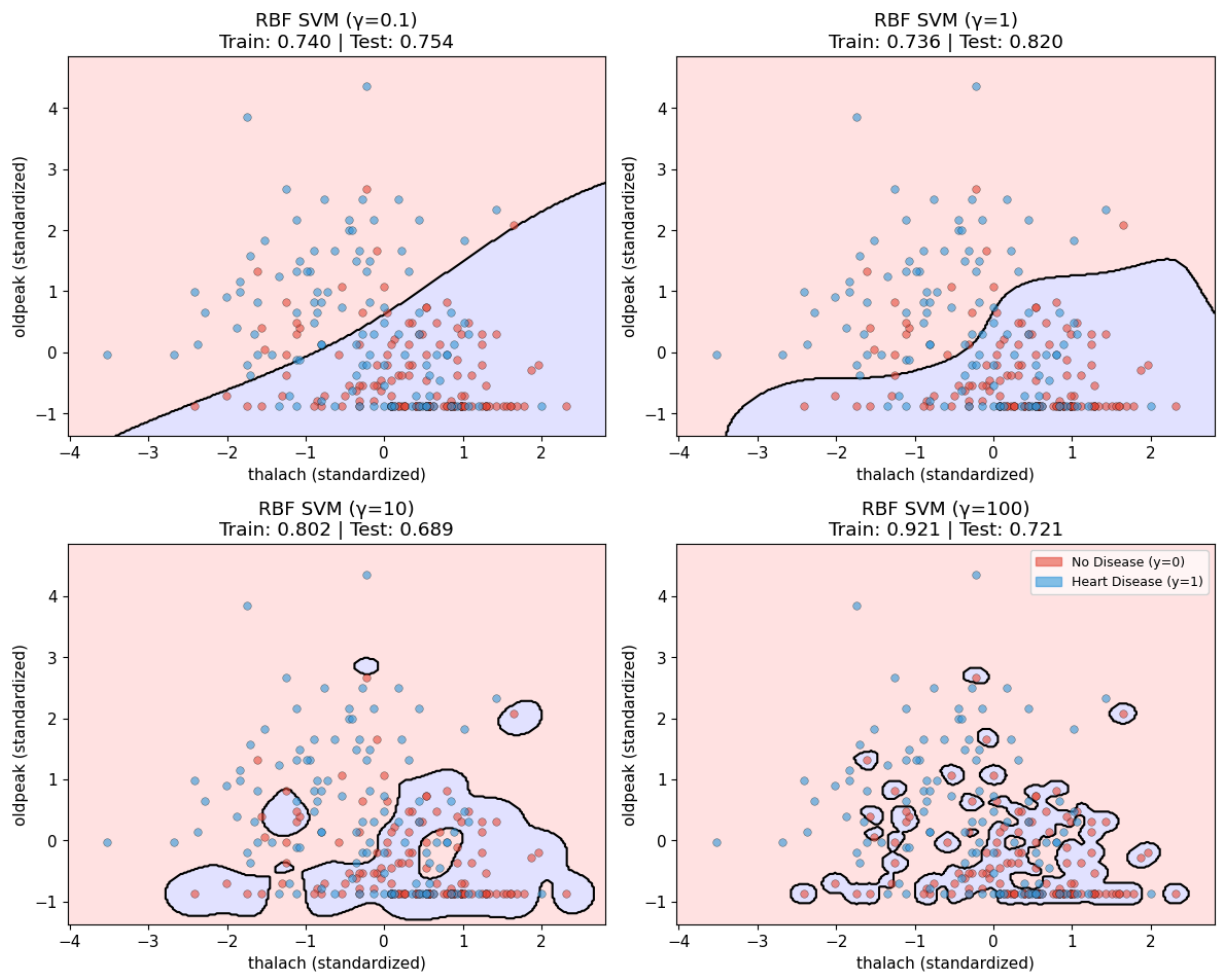
 $\gamma=0.1$ : The kernel decays very slowly – training points far apart still have
high kernel similarity – so each training point influences nearly the entire
input space. The boundary looks almost linear, and test accuracy is modest.

 $\gamma=1$ : Moderate locality. The boundary curves to match the data distribution
without overfitting badly. This is typically the sweet spot.

 $\gamma=10$ : The kernel decays quickly, making the classifier local. The boundary
becomes irregular, with small islands forming around training clusters.

 $\gamma=100$ : Severe overfitting.  $d_k$  saturates at 2 for any moderately separated
pair, so each training point only 'sees' its immediate neighbourhood.
Training accuracy is near perfect but test accuracy collapses – the boundary
is a patchwork of tiny decision regions around individual training points.
""")
```

RBF SVM Decision Boundaries (C=1.0)



Best test accuracy: $\gamma = 1$ (train=0.736, test=0.820)
 Most overfit: $\gamma = 100$ (train=0.921, test=0.721)

Connection to Problem 2(b)(iii):

The feature-space distance is $d_k(x, z) = 2(1 - \exp(-\gamma ||x - z||^2))$.

$\gamma=0.1$: The kernel decays very slowly – training points far apart still have high kernel similarity – so each training point influences nearly the entire input space. The boundary looks almost linear, and test accuracy is modest.

$\gamma=1$: Moderate locality. The boundary curves to match the data distribution without overfitting badly. This is typically the sweet spot.

$\gamma=10$: The kernel decays quickly, making the classifier local. The boundary becomes irregular, with small islands forming around training clusters.

$\gamma=100$: Severe overfitting. d_k saturates at 2 for any moderately separated pair, so each training point only 'sees' its immediate neighbourhood. Training accuracy is near perfect but test accuracy collapses – the boundary is a patchwork of tiny decision regions around individual training points.

(e) Linking Back to Theory

(e)(i) Loss Does Not Change the Boundary Shape

```
In [14]: # Logistic regression on the same two standardized features
lr = LogisticRegression(C=1.0, max_iter=1000)
lr.fit(X_train_std, y_train)

lr_train_acc = accuracy_score(y_train, lr.predict(X_train_std))
lr_test_acc = accuracy_score(y_test, lr.predict(X_test_std))

print(f"Logistic Regression: train={lr_train_acc:.4f}, test={lr_test_acc:.4f}")
print(f"Linear SVM: train={train_acc_linear:.4f}, test={test_acc_linear:.4f}")
```

Logistic Regression: train=0.7231, test=0.8033
 Linear SVM: train=0.7231, test=0.7705

```
In [15]: fig, ax = plt.subplots(figsize=(8, 6))

# Background: Linear SVM regions
x_min = X_train_std[:, 0].min() - 0.5
x_max = X_train_std[:, 0].max() + 0.5
y_min = X_train_std[:, 1].min() - 0.5
y_max = X_train_std[:, 1].max() + 0.5

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))
grid = np.c_[xx.ravel(), yy.ravel()]

Z_svm = svc_linear.predict(grid).reshape(xx.shape)
```

```

ax.contourf(xx, yy, Z_svm, alpha=0.15, cmap='bwr', levels=[-0.5, 0.5, 1.5])

# Linear SVM boundary
w_svm = svc_linear.coef_[0]
b_svm = svc_linear.intercept_[0]
x_vals = np.linspace(x_min, x_max, 300)
if abs(w_svm[1]) > 1e-10:
    db_svm = (-w_svm[0] * x_vals - b_svm) / w_svm[1]
    ax.plot(x_vals, db_svm, 'b-', lw=2.5, label='Linear SVM boundary')

# Logistic Regression boundary
w_lr = lr.coef_[0]
b_lr = lr.intercept_[0]
if abs(w_lr[1]) > 1e-10:
    db_lr = (-w_lr[0] * x_vals - b_lr) / w_lr[1]
    ax.plot(x_vals, db_lr, 'r--', lw=2.5, label='Logistic Regression boundary')

# Training points
for label, color in [(0, '#e74c3c'), (1, '#3498db')]:
    mask = y_train == label
    ax.scatter(X_train_std[mask, 0], X_train_std[mask, 1],
               c=color, alpha=0.5, edgecolors='k', linewidths=0.3, s=30, zorder=3)

ax.set_xlim(x_min, x_max)
ax.set_ylim(y_min, y_max)
ax.set_xlabel('thalach (standardized)')
ax.set_ylabel('oldpeak (standardized)')
ax.set_title('Linear SVM vs. Logistic Regression Decision Boundaries')
ax.legend(fontsize=10)

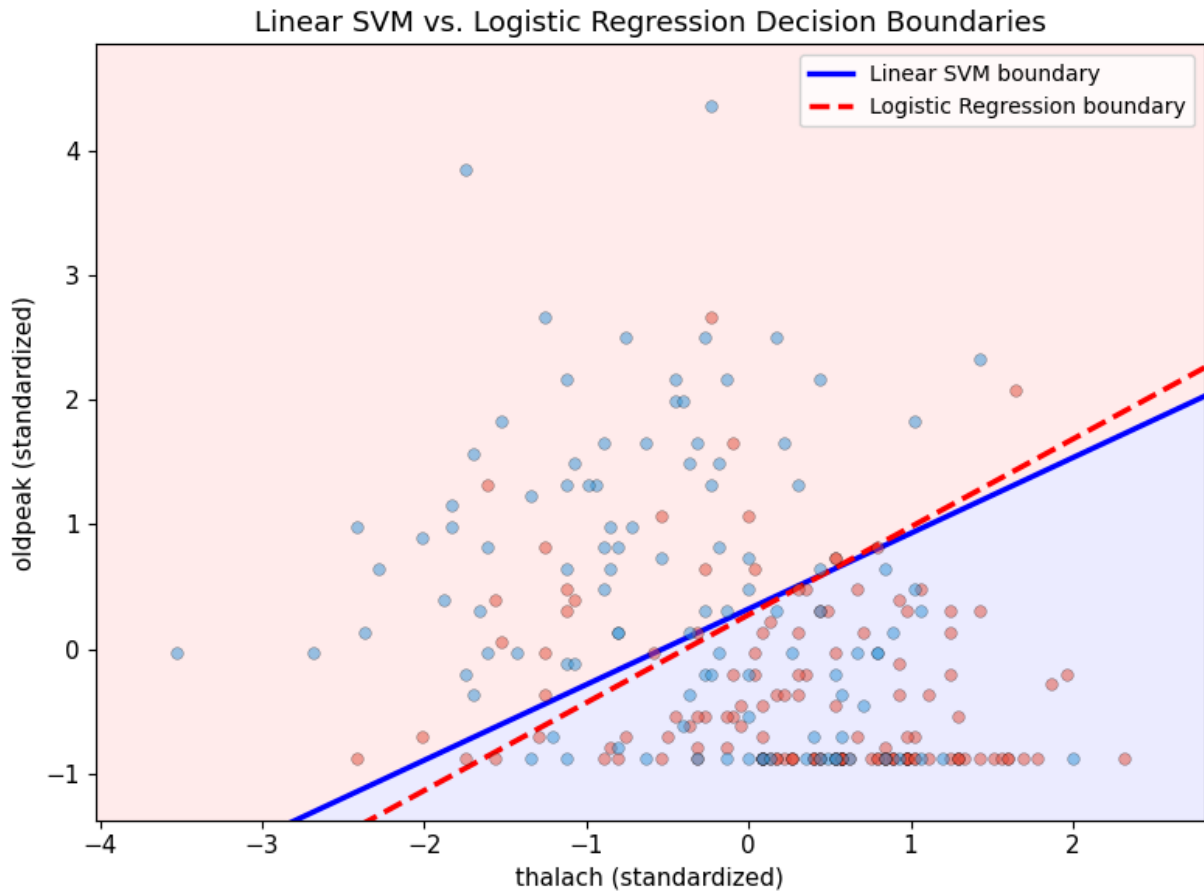
plt.tight_layout()
plt.savefig('p4_e_i_comparison.png', bbox_inches='tight')
plt.show()

print(f"\nLinear SVM weights:          w = {w_svm}, b = {b_svm:.4f}")
print(f"Logistic Regression weights:  w = {w_lr}, b = {b_lr:.4f}")

print("""
Analysis:
    Both boundaries are straight lines in the feature space – confirming Problem 1(f)
    the form of the decision boundary (linear in x) is determined by the model class,
    not by the choice of loss function.

    They do NOT coincide exactly. This is expected: the two methods optimize different
    objectives. The linear SVM maximizes the geometric margin (finding the hyperplane
    that is farthest from both classes) using hinge loss, which only cares about
    examples within the margin. Logistic regression minimizes log-loss, which receives
    gradient contributions from every training point (as shown in Problem 1(d)).
    Because logistic loss never goes to zero, even well-classified points gently pull
    on the boundary, leading to a different orientation than the max-margin SVM.
""")

```



Linear SVM weights: $w = [-0.47573091 \quad 0.78303535]$, $b = -0.2522$

Logistic Regression weights: $w = [-0.60810388 \quad 0.86173062]$, $b = -0.2372$

Analysis:

Both boundaries are straight lines in the feature space – confirming Problem 1(f): the form of the decision boundary (linear in x) is determined by the model class, not by the choice of loss function.

They do NOT coincide exactly. This is expected: the two methods optimize different objectives. The linear SVM maximizes the geometric margin (finding the hyperplane that is farthest from both classes) using hinge loss, which only cares about examples within the margin. Logistic regression minimizes log-loss, which receives gradient contributions from every training point (as shown in Problem 1(d)). Because logistic loss never goes to zero, even well-classified points gently pull on the boundary, leading to a different orientation than the max-margin SVM.

(e)(ii) Kernelized Prediction Vanishes Far From Data

```
In [16]: # RBF SVM with gamma=1
svc_rbf_1 = rbf_svcs[1]

# Far-away test point: (10, 10) in standardized coordinates
x_star = np.array([[10.0, 10.0]])

# Manual computation of  $h(x^*) = \sum_{n \in SV} \alpha_n * y_n * k(x_n, x^*) + b$ 
# In sklearn: dual_coef_ stores  $\alpha_n * y_n$  for each support vector
alpha_y = svc_rbf_1.dual_coef_[0] # shape: (n_sv,)
```



```

sv      = svc_rbf_1.support_vectors_ # shape: (n_sv, 2)
b_rbf   = svc_rbf_1.intercept_[0]
gamma   = 1.0

# RBF kernel:  $k(x_n, x^*) = \exp(-\gamma \|x_n - x^*\|^2)$ 
sq_dists = np.sum((sv - x_star)** 2, axis=1)
k_vals   = np.exp(-gamma * sq_dists)

h_star_kernel_sum = np.dot(alpha_y, k_vals)
h_star = h_star_kernel_sum + b_rbf

print(f"RBF SVM ( $\gamma=1$ ) prediction at  $x^* = (10, 10)$ :")
print(f"  Kernel sum  $\sum \alpha_n y_n k(x_n, x^*) = \{h\_star\_kernel\_sum:.6f\}")
print(f"  Intercept  $b = \{b\_rbf:.6f\}")
print(f"   $h(x^*) = \text{kernel sum} + b = \{h\_star:.6f\}")
print(f"   $|b| = \{abs(b\_rbf):.6f\}")
print(f"   $\max |k(x_n, x^*)| = \{k\_vals.max():.2e\}$  (essentially 0)")

# Compare with sklearn decision_function for sanity check
h_sklearn = svc_rbf_1.decision_function(x_star)[0]
print(f"  sklearn decision_function( $x^*$ ) =  $\{h\_sklearn:.6f\}$  (cross-check)")

print(f"""
Interpretation:
  The kernel values  $k(x_n, x^*) = \exp(-\gamma \|x_n - x^*\|^2)$  are all essentially zero
  when  $x^* = (10, 10)$  is many standard deviations from every training point.
  Therefore, the kernel sum vanishes and  $h(x^*) \approx b$  (the intercept only).

  This confirms Problem 2(d): an RBF-kernel classifier of the form
   $h(x) = \sum \alpha_n k(x_n, x) + b$  collapses to just  $b$  when  $x$  is far from all
  training points. The classifier has virtually no meaningful confidence
  about such a point – its prediction is driven entirely by the bias term
  rather than by any actual evidence from the training data.
""")$$$$ 
```

```

RBF SVM ( $\gamma=1$ ) prediction at  $x^* = (10, 10)$ :
  Kernel sum  $\sum \alpha_n y_n k(x_n, x^*) = 0.000000$ 
  Intercept  $b = 0.236702$ 
   $h(x^*) = \text{kernel sum} + b = 0.236702$ 
   $|b| = 0.236702$ 
   $\max |k(x_n, x^*)| = 3.26e-58$  (essentially 0)
  sklearn decision_function( $x^*$ ) = 0.236702 (cross-check)

```

Interpretation:

The kernel values $k(x_n, x^*) = \exp(-\gamma \|x_n - x^*\|^2)$ are all essentially zero when $x^* = (10, 10)$ is many standard deviations from every training point. Therefore, the kernel sum vanishes and $h(x^*) \approx b$ (the intercept only).

This confirms Problem 2(d): an RBF-kernel classifier of the form $h(x) = \sum \alpha_n k(x_n, x) + b$ collapses to just b when x is far from all training points. The classifier has virtually no meaningful confidence about such a point – its prediction is driven entirely by the bias term rather than by any actual evidence from the training data.

Summary Table

```
In [17]: summary_rows = [
    ('Linear SVM (C=1)',      train_acc_linear, test_acc_linear),
    ('Logistic Regression (C=1)', lr_train_acc,      lr_test_acc),
]
for d in degrees:
    tr, te = poly_results[d]
    summary_rows.append((f'Poly SVM (degree={d}, C=1)', tr, te))
for g in gammas:
    tr, te = rbf_results[g]
    summary_rows.append((f'RBF SVM (γ={g}, C=1)', tr, te))

print(f"{'Model':<35} {'Train Acc':<12} {'Test Acc':<12}")
print("-" * 59)
for name, tr, te in summary_rows:
    print(f"{'name':<35} {tr:.4f}      {te:.4f}")
```

Model	Train Acc	Test Acc
Linear SVM (C=1)	0.7231	0.7705
Logistic Regression (C=1)	0.7231	0.8033
Poly SVM (degree=2, C=1)	0.7190	0.7213
Poly SVM (degree=3, C=1)	0.7273	0.7541
Poly SVM (degree=5, C=1)	0.7355	0.7541
RBF SVM (γ=0.1, C=1)	0.7397	0.7541
RBF SVM (γ=1, C=1)	0.7355	0.8197
RBF SVM (γ=10, C=1)	0.8017	0.6885
RBF SVM (γ=100, C=1)	0.9215	0.7213

Problem 5: Implementing SVMs from Scratch — Hinge vs. Logistic Loss

Dataset: HTRU2 — Predicting a Pulsar Star

Tools: NumPy (all model fitting), pandas (I/O), matplotlib/seaborn (plots)

Constraint: No `sklearn.svm` permitted

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display

plt.rcParams.update({'figure.dpi': 110, 'font.size': 11})
sns.set_style('whitegrid')
```

Part (a): EDA and Preprocessing

(a)(i) Shape and Class Balance

```
In [2]: # Load dataset
df = pd.read_csv('HW_2_Datasets/HW_2_Datasets/HW_5/pulsar_data_train.csv')
df.columns = df.columns.str.strip() # Remove leading/trailing whitespace from columns

# Rename columns to concise identifiers
col_map = {
    'Mean of the integrated profile': 'Mean_Integrated',
    'Standard deviation of the integrated profile': 'SD_Integrated',
    'Excess kurtosis of the integrated profile': 'EK_Integrated',
    'Skewness of the integrated profile': 'Skewness_Integrated',
    'Mean of the DM-SNR curve': 'Mean_DMSNR',
    'Standard deviation of the DM-SNR curve': 'SD_DMSNR',
    'Excess kurtosis of the DM-SNR curve': 'EK_DMSNR',
    'Skewness of the DM-SNR curve': 'Skewness_DMSNR',
}
df.rename(columns=col_map, inplace=True)

FEATURE_COLS = list(col_map.values())
TARGET_COL = 'target_class'

# Drop rows with any missing values
n_before = len(df)
df.dropna(inplace=True)
df.reset_index(drop=True, inplace=True)
print(f'Rows before drop: {n_before} | After drop: {len(df)} | Dropped: {n_before - len(df)}')
```

```
print(f'Shape: {df.shape}\n')

display(df.describe())

pulsar_frac = df[TARGET_COL].mean()
print(f'\nFraction with target_class = 1 (pulsar): {pulsar_frac:.4f} ({100*pulsar_frac:.2f}%)')
print(f'Trivial "always predict 0" accuracy: {100*(1-pulsar_frac):.2f}%')
print('')
Why accuracy alone is misleading:
    The dataset is heavily class-imbalanced (~90% non-pulsar, ~10% pulsar).
    A trivial classifier that always outputs class 0 achieves ~90% accuracy
    without learning anything useful about pulsars. Metrics like precision,
    recall, F1, or AUC-PR better capture performance on the rare positive class.
    ''')
```

Rows before drop: 12528 | After drop: 9273 | Dropped: 3255
 Shape: (9273, 9)

	Mean_Integrated	SD_Integrated	EK_Integrated	Skewness_Integrated	Mean_DMSNR
count	9273.000000	9273.000000	9273.000000	9273.000000	9273.000000
mean	111.133750	46.514053	0.479546	1.790673	12.735359
std	25.686405	6.781320	1.071155	6.286567	29.766782
min	6.187500	24.772042	-1.738021	-1.791886	0.213211
25%	100.984375	42.398382	0.024497	-0.188021	1.910535
50%	115.234375	46.895026	0.223858	0.202884	2.797659
75%	127.328125	51.001785	0.472066	0.930710	5.459866
max	189.734375	91.808628	8.069522	68.101622	211.948997

◀  ▶
 Fraction with target_class = 1 (pulsar): 0.0917 (9.17%)
 Trivial "always predict 0" accuracy: 90.83%

Why accuracy alone is misleading:
 The dataset is heavily class-imbalanced (~90% non-pulsar, ~10% pulsar).
 A trivial classifier that always outputs class 0 achieves ~90% accuracy
 without learning anything useful about pulsars. Metrics like precision,
 recall, F1, or AUC-PR better capture performance on the rare positive class.

(a)(ii) Feature Distributions by Class

```
In [3]: pulsars      = df[df[TARGET_COL] == 1]
        non_pulsars = df[df[TARGET_COL] == 0]

        feature_labels = [
            'Mean Integrated', 'SD Integrated', 'EK Integrated', 'Skewness Integrated',
            'Mean DM-SNR',     'SD DM-SNR',     'EK DM-SNR',     'Skewness DM-SNR'
        ]
```



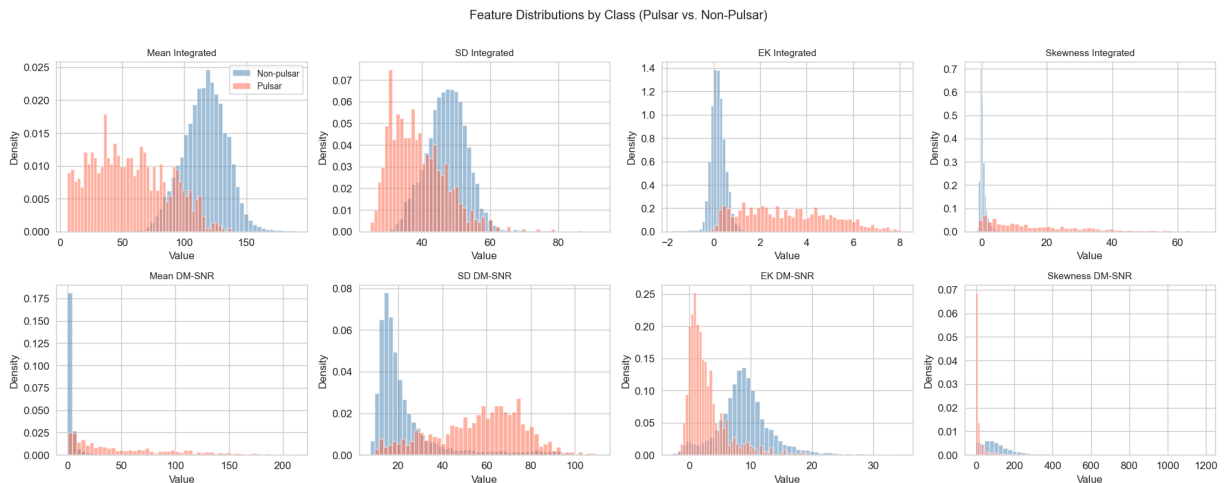
```
fig, axes = plt.subplots(2, 4, figsize=(18, 7))
axes = axes.flatten()

for i, (col, label) in enumerate(zip(FEATURE_COLS, feature_labels)):
    ax = axes[i]
    ax.hist(non_pulsars[col].dropna(), bins=50, alpha=0.5,
           color='steelblue', label='Non-pulsar', density=True)
    ax.hist(pulsars[col].dropna(), bins=50, alpha=0.5,
           color='tomato', label='Pulsar', density=True)
    ax.set_title(label, fontsize=10)
    ax.set_xlabel('Value')
    ax.set_ylabel('Density')
    if i == 0:
        ax.legend(fontsize=9)

plt.suptitle('Feature Distributions by Class (Pulsar vs. Non-Pulsar)', fontsize=13,
plt.tight_layout()
plt.show()

print('''
Observations:
    Best-separating features (clear bimodal or shift): Mean_Integrated, SD_Integrated
    Mean_DMSNR, SD_DMSNR. Pulsars tend to have lower Mean_Integrated and lower Mean_D

Relation to margin intuition (Problem 1.3):
    Features that separate classes well produce large  $|m_n| = |y_n(w^T x_n + b)|$  once
    the model is trained, because the decision hyperplane can achieve a large margin.
    Features with heavily overlapping distributions (EK_Integrated, Skewness_Integrated)
    will contribute less discriminative power and produce smaller per-example margins
''')
```



Observations:

Best-separating features (clear bimodal or shift): Mean_Integrated, SD_Integrated, Mean_DMSNR, SD_DMSNR. Pulsars tend to have lower Mean_Integrated and lower Mean_DM SNR.

Relation to margin intuition (Problem 1.3):

Features that separate classes well produce large $|m_n| = |y_n(w^T x_n + b)|$ once the model is trained, because the decision hyperplane can achieve a large margin.

Features with heavily overlapping distributions (EK_Integrated, Skewness_Integrated) will contribute less discriminative power and produce smaller per-example margins.

(a)(iii) Preprocessing

```
In [4]: # Convert labels: 0 -> -1, 1 -> +1 (SVM convention from Problem 1)
X_all = df[FEATURE_COLS].values.astype(float)
y_all = np.where(df[TARGET_COL].values == 1, 1.0, -1.0)

N = len(X_all)
print(f'Total samples: {N}')

# 60 / 20 / 20 split
np.random.seed(42)
idx = np.random.permutation(N)
n_train = int(0.60 * N)
n_val = int(0.20 * N)
# n_test = N - n_train - n_val (remainder)

train_idx = idx[:n_train]
val_idx = idx[n_train : n_train + n_val]
test_idx = idx[n_train + n_val :]

X_train, y_train = X_all[train_idx], y_all[train_idx]
X_val, y_val = X_all[val_idx], y_all[val_idx]
X_test, y_test = X_all[test_idx], y_all[test_idx]

print(f'Train: {X_train.shape} | Val: {X_val.shape} | Test: {X_test.shape}')
print(f'Train pulsar rate: {(y_train==1).mean():.3f} | '
      f'Val: {(y_val==1).mean():.3f} | Test: {(y_test==1).mean():.3f}')

# Standardize using training-set statistics ONLY
mu = X_train.mean(axis=0)
sigma = X_train.std(axis=0)

X_train_s = (X_train - mu) / sigma
X_val_s = (X_val - mu) / sigma
X_test_s = (X_test - mu) / sigma

print('\nConvention: w (shape d,) and b (scalar) are kept SEPARATE throughout.')
print('Labels are y in {-1, +1} (SVM convention).')
print('\nWhy not standardize on combined train+test?')
print(' Using test-set statistics during preprocessing leaks information about the
print(' distribution into training. This inflates estimated generalisation perform
```

```
print(' (data leakage). mu and sigma must be computed on training data only, then'
print(' applied to val and test as fixed transformations.')
```

Total samples: 9273

Train: (5563, 8) | Val: (1854, 8) | Test: (1856, 8)

Train pulsar rate: 0.091 | Val: 0.092 | Test: 0.093

Convention: w (shape d ,) and b (scalar) are kept SEPARATE throughout.

Labels are y in $\{-1, +1\}$ (SVM convention).

Why not standardize on combined train+test?

Using test-set statistics during preprocessing leaks information about the test distribution into training. This inflates estimated generalisation performance (data leakage). μ and σ must be computed on training data only, then applied to val and test as fixed transformations.

Part (b): Hinge-Loss SVM via Subgradient Descent

(b)(i) Gradient Formula

For a single example (x, y) with margin $m = y(w^T x + b)$:

$$\partial_w \ell_{\text{hinge}} = -y x \cdot \mathbf{1}[m < 1], \quad \partial_b \ell_{\text{hinge}} = -y \cdot \mathbf{1}[m < 1]$$

Full objective (mean-normalized for numerical stability with $\eta = 10^{-3}$):

$$J_{\text{hinge}}(w, b) = \frac{1}{N} \sum_{n=1}^N \max(0, 1 - m_n) + \frac{\lambda}{2} \|w\|^2$$

Subgradient of the full objective:

$$\nabla_w J_{\text{hinge}} = \frac{1}{N} \sum_{n=1}^N -y_n x_n \cdot \mathbf{1}[m_n < 1] + \lambda w, \quad \frac{\partial J_{\text{hinge}}}{\partial b} = \frac{1}{N} \sum_{n=1}^N -y_n \cdot \mathbf{1}[m_n < 1]$$

(The $\frac{1}{N}$ normalisation makes the magnitude of the gradient independent of dataset size, so the same $\eta = 10^{-3}$ is sensible.)

(b)(ii) Implementation and Training

```
In [5]: def train_hinge_svm(X, y, lam=1e-3, eta=1e-3, n_iter=2000):
        """
        Full-batch subgradient descent for the hinge-loss SVM.
        Objective: J = (1/N)*sum_n max(0,1-m_n) + (lam/2)*||w||^2
        Returns: w, b, loss history
        """
        N, d = X.shape
        w = np.zeros(d)
        b = 0.0
        losses = np.empty(n_iter)
```

```

for t in range(n_iter):
    margins = y * (X @ w + b)                # (N,)
    hinge   = np.maximum(0.0, 1.0 - margins)  # per-example loss
    losses[t] = hinge.mean() + 0.5 * lam * np.dot(w, w)

    active = (margins < 1.0).astype(float)    # 1 where subgradient is
    grad_w = -(active * y) @ X / N + lam * w
    grad_b = -(active * y).sum() / N

    w -= eta * grad_w
    b -= eta * grad_b

return w, b, losses

LAM   = 1e-3
ETA   = 1e-3
N_ITER = 2000

w_hinge, b_hinge, losses_hinge = train_hinge_svm(
    X_train_s, y_train, lam=LAM, eta=ETA, n_iter=N_ITER
)

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(losses_hinge, color='steelblue', linewidth=1.2)
ax.set_xlabel('Iteration')
ax.set_ylabel('Training Loss')
ax.set_title('Hinge-Loss SVM: Training Loss vs. Iteration')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('Is the loss monotonically decreasing?')
mono = np.all(np.diff(losses_hinge) <= 1e-12)
print(f' Monotone: {mono}')
print('')
Subgradient descent is NOT guaranteed to decrease the objective at every step.
Unlike a true gradient, a subgradient at a non-differentiable point (m=1) is not
necessarily a descent direction, so individual steps can increase the loss. In prac
the loss trend is downward but with local oscillations near the kink at m=1.
''')

```



Is the loss monotonically decreasing?

Monotone: True

Subgradient descent is NOT guaranteed to decrease the objective at every step. Unlike a true gradient, a subgradient at a non-differentiable point ($m=1$) is not necessarily a descent direction, so individual steps can increase the loss. In practice the loss trend is downward but with local oscillations near the kink at $m=1$.

(b)(iii) Who Is Actually Contributing?

```
In [6]: def accuracy(X, y, w, b):
        return np.mean(np.sign(X @ w + b) == y)

train_acc_hinge = accuracy(X_train_s, y_train, w_hinge, b_hinge)
val_acc_hinge   = accuracy(X_val_s,   y_val,   w_hinge, b_hinge)
print(f'Hinge SVM | Train acc: {train_acc_hinge:.4f} | Val acc: {val_acc_hinge:.4f}')

# Margins on training set
margins_hinge_train = y_train * (X_train_s @ w_hinge + b_hinge)

# m_n >= 1 <=> zero subgradient
zero_mask = margins_hinge_train >= 1.0
n_zero_total = zero_mask.sum()
n_zero_pulsar = (zero_mask & (y_train == 1)).sum()
n_zero_nonp   = (zero_mask & (y_train == -1)).sum()

active_mask = ~zero_mask
n_act_total = active_mask.sum()
n_act_pulsar = (active_mask & (y_train == 1)).sum()
n_act_nonp   = (active_mask & (y_train == -1)).sum()

N_tr = len(y_train)
print(f'\nTraining set size: {N_tr}')
print(f' Pulsars (y=+1): {(y_train==1).sum()} | Non-pulsars (y=-1): {(y_train==-1).sum()}')
print(f'\nExamples with m_n >= 1 (zero subgradient): {n_zero_total} / {N_tr} ({100*n_zero_total/N_tr:.1f}%)')
print(f' Breakdown | Non-pulsars: {n_zero_nonp} | Pulsars: {n_zero_pulsar}')
print(f'\nActive examples (pulling on the boundary): {n_act_total} / {N_tr} ({100*n_act_total/N_tr:.1f}%)')
```



```
print(f' Breakdown | Non-pulsars: {n_act_nonp} | Pulsars: {n_act_pulsar}')

n_pulsars_tr = (y_train == 1).sum()
n_nonpuls_tr = (y_train == -1).sum()
print(f'\nActive rate by class:')
print(f' Pulsars: {n_act_pulsar}/{n_pulsars_tr} = {100*n_act_pulsar/n_pulsars_tr:.1f}%')
print(f' Non-pulsars: {n_act_nonp}/{n_nonpuls_tr} = {100*n_act_nonp/n_nonpuls_tr:.1f}%')
print('')
Interpretation:
    Most of the absolute active count sits in the majority class (non-pulsars) simply
    because it is far larger. However, the per-class active RATE for pulsars may be
    higher, meaning that proportionally more pulsars are still inside the margin and
    pulling on the boundary. This makes sense: pulsars are rarer, the model has seen
    fewer of them, so they are harder to push outside the margin.
''')
```

Hinge SVM | Train acc: 0.9700 | Val acc: 0.9682

Training set size: 5563

Pulsars (y=+1): 506 | Non-pulsars (y=-1): 5057

Examples with $m_n \geq 1$ (zero subgradient): 4270 / 5563 (76.8%)

Breakdown | Non-pulsars: 4045 | Pulsars: 225

Active examples (pulling on the boundary): 1293 / 5563 (23.2%)

Breakdown | Non-pulsars: 1012 | Pulsars: 281

Active rate by class:

Pulsars: 281/506 = 55.5%

Non-pulsars: 1012/5057 = 20.0%

Interpretation:

Most of the absolute active count sits in the majority class (non-pulsars) simply because it is far larger. However, the per-class active RATE for pulsars may be higher, meaning that proportionally more pulsars are still inside the margin and pulling on the boundary. This makes sense: pulsars are rarer, the model has seen fewer of them, so they are harder to push outside the margin.

Part (c): Logistic-Loss SVM

(c)(i) Gradient Formula

For a single example (x, y) with margin $m = y(w^T x + b)$, using $\sigma(z) = 1/(1 + e^{-z})$:

$$\nabla_w \ell_{\log} = -\sigma(-m) y x, \quad \frac{\partial \ell_{\log}}{\partial b} = -\sigma(-m) y$$

Full mean-normalised objective:

$$J_{\log}(w, b) = \frac{1}{N} \sum_{n=1}^N \log(1 + e^{-m_n}) + \frac{\lambda}{2} \|w\|^2$$

Gradient of full objective:

$$\nabla_w J_{\log} = \frac{1}{N} \sum_{n=1}^N -\sigma(-m_n) y_n x_n + \lambda w, \quad \frac{\partial J_{\log}}{\partial b} = \frac{1}{N} \sum_{n=1}^N -\sigma(-m_n) y_n$$

(c)(ii) Implementation and Training

```
In [7]: def stable_sigmoid(z):
    """Numerically stable sigmoid: avoids overflow for large |z|. """
    return np.where(
        z >= 0,
        1.0 / (1.0 + np.exp(-z)),
        np.exp(z) / (1.0 + np.exp(z))
    )

def train_logistic_svm(X, y, lam=1e-3, eta=1e-3, n_iter=2000):
    """
    Full-batch gradient descent for the logistic-loss SVM.
    Objective: J = (1/N)*sum_n logaddexp(0, -m_n) + (lam/2)*||w||^2
    Uses np.logaddexp(0, -m) for numerically stable log(1 + exp(-m)).
    Returns: w, b, loss history
    """
    N, d = X.shape
    w = np.zeros(d)
    b = 0.0
    losses = np.empty(n_iter)

    for t in range(n_iter):
        margins = y * (X @ w + b) # (N,)
        losses[t] = np.logaddexp(0.0, -margins).mean() + 0.5 * lam * np.dot(w, w)

        sig_neg_m = stable_sigmoid(-margins) # sigma(-m_n)
        grad_w = -(sig_neg_m * y) @ X / N + lam * w
        grad_b = -(sig_neg_m * y).sum() / N

        w -= eta * grad_w
        b -= eta * grad_b

    return w, b, losses

w_log, b_log, losses_log = train_logistic_svm(
    X_train_s, y_train, lam=LAM, eta=ETA, n_iter=N_ITER
)

# Plot both loss curves on log-scaled y-axis
fig, ax = plt.subplots(figsize=(9, 4))
ax.semilogy(losses_hinge, color='steelblue', linewidth=1.2, label='Hinge Loss')
ax.semilogy(losses_log, color='tomato', linewidth=1.2, label='Logistic Loss')
ax.set_xlabel('Iteration')
ax.set_ylabel('Training Loss (log scale)')
ax.set_title('Training Loss Curves: Hinge vs. Logistic SVM')
```

```
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('''
Convergence comparison:
    Logistic loss converges more smoothly because it is everywhere differentiable;
    gradient descent always has a valid, consistent descent direction. The hinge loss
    has a non-differentiable kink at m=1, so subgradient steps can occasionally
    overshoot, causing oscillations visible as a noisier loss curve.
    At eta=1e-3 both converge within 2000 iterations on this dataset.
''')
```



Convergence comparison:

Logistic loss converges more smoothly because it is everywhere differentiable; gradient descent always has a valid, consistent descent direction. The hinge loss has a non-differentiable kink at $m=1$, so subgradient steps can occasionally overshoot, causing oscillations visible as a noisier loss curve. At $\eta=1e-3$ both converge within 2000 iterations on this dataset.

(c)(iii) Is Any Example Ever Exactly Inactive?

```
In [8]: train_acc_log = accuracy(X_train_s, y_train, w_log, b_log)
val_acc_log = accuracy(X_val_s, y_val, w_log, b_log)
print(f'Logistic SVM | Train acc: {train_acc_log:.4f} | Val acc: {val_acc_log:.4f}')

# Per-example gradient norm: ||nabla_w \ell_n||_2 = \sigma(-m_n) * ||x_n||_2
margins_log_train = y_train * (X_train_s @ w_log + b_log)
sig_neg_m_train = stable_sigmoid(-margins_log_train)
x_norms_train = np.linalg.norm(X_train_s, axis=1)
grad_norms_log = sig_neg_m_train * x_norms_train

n_exact_zero = int((grad_norms_log == 0.0).sum())
n_below_1e6 = int((grad_norms_log < 1e-6).sum())

print(f'\nLogistic SVM - per-example gradient norm analysis:')
print(f' Exactly zero: {n_exact_zero}')
print(f' Below 1e-6: {n_below_1e6}')
```

```
print(f'\nHinge SVM – examples with zero subgradient (m_n >= 1): {n_zero_total}')
print('')
Conclusion:
    Logistic loss NEVER assigns exactly zero gradient to any training example,
    confirming Problem 1(4b). For any finite margin m,  $\sigma(-m) = 1/(1+\exp(m)) > 0$ ,
    so  $\|\nabla_{\mathbf{w}} \ell_n\|_2 = \sigma(-m_n) \|\mathbf{x}_n\|_2 > 0$  always.
    Only in floating-point arithmetic (sigma underflowing to 0 for very large m)
    would a near-zero count appear; those examples are extremely well classified.
    By contrast, hinge loss has a hard zero region for all  $m \geq 1$ .
    ''')
```

Logistic SVM | Train acc: 0.9644 | Val acc: 0.9698

Logistic SVM – per-example gradient norm analysis:

Exactly zero: 0
Below 1e-6: 0

Hinge SVM – examples with zero subgradient (m_n >= 1): 4270

Conclusion:

Logistic loss NEVER assigns exactly zero gradient to any training example, confirming Problem 1(4b). For any finite margin m, $\sigma(-m) = 1/(1+\exp(m)) > 0$, so $\|\nabla_{\mathbf{w}} \ell_n\|_2 = \sigma(-m_n) \|\mathbf{x}_n\|_2 > 0$ always. Only in floating-point arithmetic (sigma underflowing to 0 for very large m) would a near-zero count appear; those examples are extremely well classified. By contrast, hinge loss has a hard zero region for all $m \geq 1$.

Part (d): Margin vs. Gradient Magnitude — Empirical Check of Problem 1(3)

```
In [9]: # Logistic gradient norm:  $\sigma(-m_n) * \|\mathbf{x}_n\|_2$ 
# (already computed above as grad_norms_Log and margins_Log_train)

# Hinge gradient norm:  $\|\mathbf{x}_n\|_2 * 1[m_n < 1]$ 
grad_norms_hinge_train = x_norms_train * (margins_hinge_train < 1.0).astype(float)

# Subsample for plotting readability
rng_plot = np.random.default_rng(seed=1)
n_plot = min(4000, len(y_train))
idx_plot = rng_plot.choice(len(y_train), size=n_plot, replace=False)

fig, ax = plt.subplots(figsize=(11, 6))

ax.scatter(
    margins_log_train[idx_plot], grad_norms_log[idx_plot],
    s=4, alpha=0.35, color='tomato', label='Logistic  $|\nabla \ell_n|$  (scatter)'
)
ax.scatter(
    margins_hinge_train[idx_plot], grad_norms_hinge_train[idx_plot],
    s=4, alpha=0.25, color='steelblue', label='Hinge  $|\nabla \ell_n|$  (scatter)'
)
```

```

# Theoretical overlay using median feature norm as reference
x_norm_ref = float(np.median(x_norms_train))
m_range     = np.linspace(-6, 8, 600)
logistic_curve = stable_sigmoid(-m_range) * x_norm_ref
hinge_curve    = x_norm_ref * (m_range < 1.0).astype(float)

ax.plot(m_range, logistic_curve, 'r-', linewidth=2.0,
        label=f'Logistic curve  $\sigma(-m) \cdot \|x\|$  ( $\|x\|=\{x\_norm\_ref:.2f\}$ )')
ax.plot(m_range, hinge_curve, 'b-', linewidth=2.0,
        label=f'Hinge step  $\|x\| \cdot 1[m < 1]$  ( $\|x\|=\{x\_norm\_ref:.2f\}$ )')

ax.axvline(x=1, color='gray', linestyle='--', alpha=0.6, label='m = 1 (margin bound)')
ax.axvline(x=0, color='gray', linestyle=':', alpha=0.5, label='m = 0 (decision boundary)')

ax.set_xlabel('Margin  $m_n = y_n(w^T x_n + b)$ ', fontsize=12)
ax.set_ylabel('Per-example gradient norm  $\|\nabla_w \ell_n\|_2$ ', fontsize=12)
ax.set_title('Margin vs. Gradient Magnitude: Logistic vs. Hinge Loss', fontsize=13)
ax.set_xlim(-6, 8)
ax.legend(loc='upper right', markerscale=4, fontsize=9)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('')
Three regimes confirmed (Problem 1.3):
    m >> 0 (confidently correct): logistic gradient  $\rightarrow 0$  exponentially via  $\sigma(-m)$ ;
        hinge gradient = 0 (hard cut-off at  $m=1$ ). Both eventually silent, but
        logistic decays gradually while hinge switches sharply.

    m  $\sim$  0 (near decision boundary): logistic gradient  $\sim 0.5 \cdot \|x\|$ ; these points
        contribute the most per-example gradient – they are right on the fence.

    m << 0 (confidently wrong): logistic saturates at  $\sim \|x\|$  ( $\sigma(-m) \rightarrow 1$  but  $\leq 1$ );
        hinge also equals  $\|x\|$ . Both are at maximum contribution, but logistic
        is bounded – there is no way for a single example to dominate the update
        beyond its own feature norm.

Key contrast (Problem 1.5):
    Hinge is a binary "you matter / you don't" switch at  $m=1$ : gradient is either  $\|x\|$ 
    or 0 with no intermediate state. Logistic is a smooth sigmoidal interpolation that
    never fully silences an example, giving a gentler and more robust gradient signal
    ''')

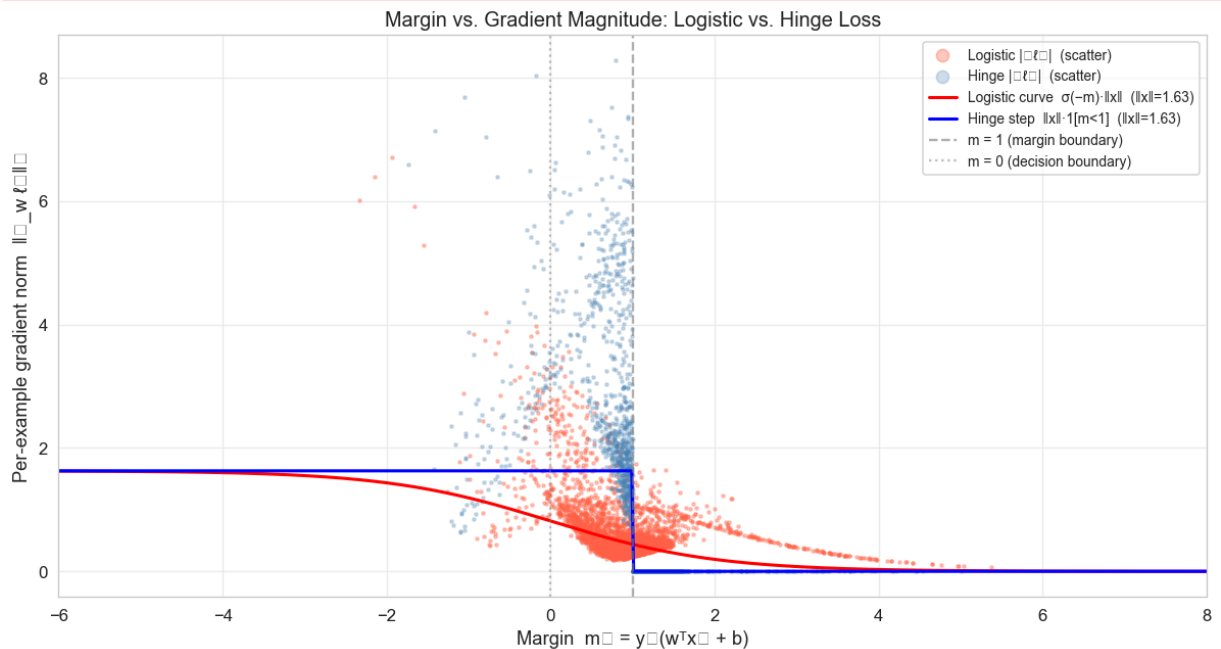
```



```

C:\Users\HarryMyers\AppData\Local\Temp\ipykernel_23664\3315161303.py:43: UserWarning:
Glyph 8345 (\N{LATIN SUBSCRIPT SMALL LETTER N}) missing from font(s) Arial.
plt.tight_layout()
C:\Users\HarryMyers\AppData\Local\Temp\ipykernel_23664\3315161303.py:43: UserWarning:
Glyph 8711 (\N{NABLA}) missing from font(s) Arial.
plt.tight_layout()
C:\Users\HarryMyers\AppData\Local\Temp\ipykernel_23664\3315161303.py:43: UserWarning:
Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.
plt.tight_layout()
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8345 (\N{LATIN SUBSCRIPT SMALL LETTER N}) missing from font(s) Arial.
fig.canvas.print_figure(bytes_io, **kw)
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8711 (\N{NABLA}) missing from font(s) Arial.
fig.canvas.print_figure(bytes_io, **kw)
C:\Users\HarryMyers\AppData\Roaming\Python\Python314\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.
fig.canvas.print_figure(bytes_io, **kw)

```



Three regimes confirmed (Problem 1.3):

$m \gg 0$ (confidently correct): logistic gradient $\rightarrow 0$ exponentially via $\sigma(-m) \rightarrow 0$;

hinge gradient = 0 (hard cut-off at $m=1$). Both eventually silent, but logistic decays gradually while hinge switches sharply.

$m \sim 0$ (near decision boundary): logistic gradient $\sim 0.5 \cdot ||x||$; these points contribute the most per-example gradient – they are right on the fence.

$m \ll 0$ (confidently wrong): logistic saturates at $\sim ||x||$ ($\sigma(-m) \rightarrow 1$ but ≤ 1); hinge also equals $||x||$. Both are at maximum contribution, but logistic is bounded – there is no way for a single example to dominate the update beyond its own feature norm.

Key contrast (Problem 1.5):

Hinge is a binary "you matter / you don't" switch at $m=1$: gradient is either $||x||$ or 0 with no intermediate state. Logistic is a smooth sigmoidal interpolation that never fully silences an example, giving a gentler and more robust gradient signal.

Part (e): Robustness to Label Noise

```
In [10]: # Corrupt 5% of training labels by flipping them
np.random.seed(0)
n_corrupt = int(0.05 * len(y_train))
corrupt_idx = np.random.choice(len(y_train), size=n_corrupt, replace=False)

y_train_noisy = y_train.copy()
y_train_noisy[corrupt_idx] *= -1

print(f'Flipped {n_corrupt} labels ({100*n_corrupt/len(y_train):.1f}% of {len(y_train)})')

# Retrain both models from scratch on noisy labels
w_hinge_n, b_hinge_n, _ = train_hinge_svm(
    X_train_s, y_train_noisy, lam=LAM, eta=ETA, n_iter=N_ITER
)
w_log_n, b_log_n, _ = train_logistic_svm(
    X_train_s, y_train_noisy, lam=LAM, eta=ETA, n_iter=N_ITER
)

# Accuracy evaluated on CLEAN validation set
val_acc_hinge_n = accuracy(X_val_s, y_val, w_hinge_n, b_hinge_n)
val_acc_log_n = accuracy(X_val_s, y_val, w_log_n, b_log_n)

norm_hinge_clean = np.linalg.norm(w_hinge)
norm_hinge_noisy = np.linalg.norm(w_hinge_n)
norm_log_clean = np.linalg.norm(w_log)
norm_log_noisy = np.linalg.norm(w_log_n)

print('\n--- Robustness to Label Noise ---')
print(f'{"Model":<30} {"Clean val acc":>14} {"Noisy val acc":>14} {"Δ acc":>10}')
print(f'-' * 70)
print(f'{"Hinge SVM":<30} {val_acc_hinge:>14.4f} {val_acc_hinge_n:>14.4f} '
```

```

    f'{val_acc_hinge_n - val_acc_hinge:>+10.4f}')
print(f'{"Logistic SVM":<30} {val_acc_log:>14.4f} {val_acc_log_n:>14.4f} '
      f'{val_acc_log_n - val_acc_log:>+10.4f}')

print(f'\n{"Model":<30} {"||w|| clean":>12} {"||w|| noisy":>12} {"Δ ||w||":>10}')
print('-' * 66)
print(f'{"Hinge SVM":<30} {norm_hinge_clean:>12.4f} {norm_hinge_noisy:>12.4f} '
      f'{norm_hinge_noisy - norm_hinge_clean:>+10.4f}')
print(f'{"Logistic SVM":<30} {norm_log_clean:>12.4f} {norm_log_noisy:>12.4f} '
      f'{norm_log_noisy - norm_log_clean:>+10.4f}')

print(''''
Analysis (connecting to Problem 1.5):
  For a confidently-wrong mislabelled point ( $m \ll 0$  under flipped label),
  hinge subgradient =  $-yx$  with magnitude  $||x||$  – constant regardless of how
  wrong the label is. The logistic gradient =  $-\sigma(-m)y \cdot x$ , magnitude
   $\sigma(-m) \cdot ||x|| \leq ||x||$ . Because  $\sigma(-m) \rightarrow 1$  as  $m \rightarrow -\infty$ , both losses
  reach the same upper bound for extremely wrong points. However, during early
  training when margins are moderate, hinge already hits its full  $||x||$  contribution
  while logistic is still ramping up – making hinge more sensitive to noise
  especially in early iterations. The  $||w||$  change measures how much the
  hyperplane is dragged by the corrupt labels.
''')
```

Flipped 278 labels (5.0% of 5563 training examples)

--- Robustness to Label Noise ---

Model	Clean val acc	Noisy val acc	Δ acc
Hinge SVM	0.9682	0.9666	-0.0016
Logistic SVM	0.9698	0.9698	+0.0000

Model	$ w $ clean	$ w $ noisy	Δ $ w $
Hinge SVM	0.4488	0.3922	-0.0566
Logistic SVM	0.4533	0.3831	-0.0701

Analysis (connecting to Problem 1.5):

For a confidently-wrong mislabelled point ($m \ll 0$ under flipped label), hinge subgradient = $-yx$ with magnitude $||x||$ – constant regardless of how wrong the label is. The logistic gradient = $-\sigma(-m)y \cdot x$, magnitude $\sigma(-m) \cdot ||x|| \leq ||x||$. Because $\sigma(-m) \rightarrow 1$ as $m \rightarrow -\infty$, both losses reach the same upper bound for extremely wrong points. However, during early training when margins are moderate, hinge already hits its full $||x||$ contribution while logistic is still ramping up – making hinge more sensitive to noise especially in early iterations. The $||w||$ change measures how much the hyperplane is dragged by the corrupt labels.

Part (f): Regularization Sweep and Final Evaluation

(f)(i)–(ii) Lambda sweep on validation set

```

In [11]: LAMBDAS = [1e-4, 1e-3, 1e-2, 1e-1, 1.0, 10.0]

reg_results = []
for lam_i in LAMBDAS:
    w_i, b_i, _ = train_logistic_svm(
        X_train_s, y_train, lam=lam_i, eta=ETA, n_iter=N_ITER
    )
    tr_acc = accuracy(X_train_s, y_train, w_i, b_i)
    va_acc = accuracy(X_val_s, y_val, w_i, b_i)
    reg_results.append({'lam': lam_i, 'train': tr_acc, 'val': va_acc, 'w': w_i, 'b': b_i})
    print(f'lambda = {lam_i:.0e} | Train: {tr_acc:.4f} | Val: {va_acc:.4f}')

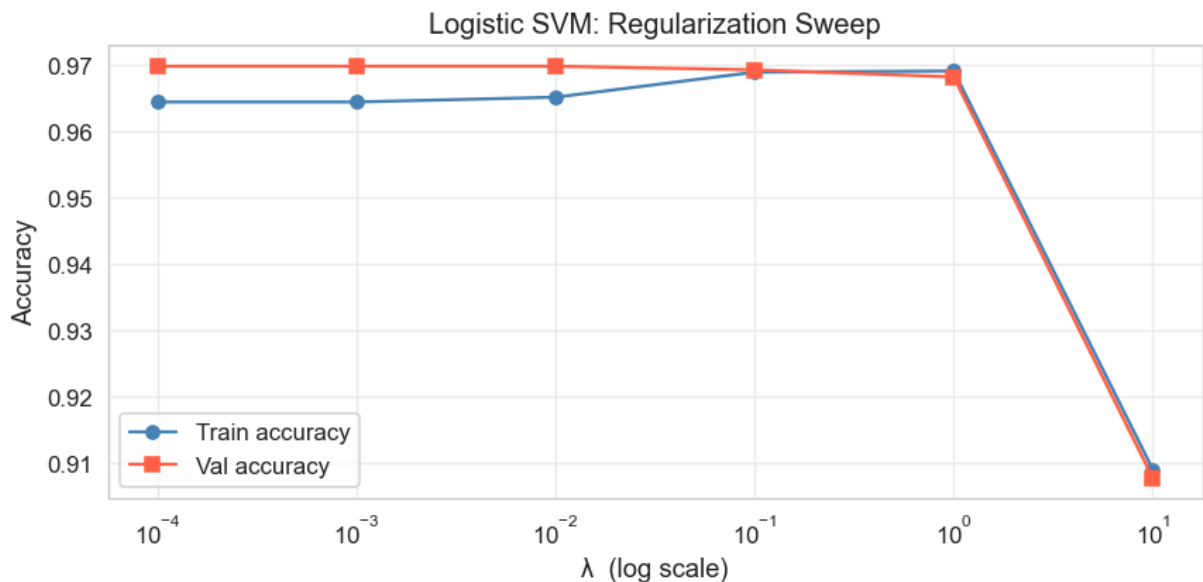
lam_vals = [r['lam'] for r in reg_results]
train_accs = [r['train'] for r in reg_results]
val_accs = [r['val'] for r in reg_results]

fig, ax = plt.subplots(figsize=(8, 4))
ax.semilogx(lam_vals, train_accs, 'o-', color='steelblue', label='Train accuracy')
ax.semilogx(lam_vals, val_accs, 's-', color='tomato', label='Val accuracy')
ax.set_xlabel('λ (log scale)', fontsize=12)
ax.set_ylabel('Accuracy', fontsize=12)
ax.set_title('Logistic SVM: Regularization Sweep', fontsize=13)
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

best_idx = int(np.argmax(val_accs))
best_lam = reg_results[best_idx]['lam']
print(f'\nBest lambda* = {best_lam:.0e} (val accuracy = {val_accs[best_idx]:.4f})')
print('Using validation (not test) to select lambda prevents test-set leakage.')

```

lambda = 1e-04	Train: 0.9644	Val: 0.9698
lambda = 1e-03	Train: 0.9644	Val: 0.9698
lambda = 1e-02	Train: 0.9651	Val: 0.9698
lambda = 1e-01	Train: 0.9689	Val: 0.9693
lambda = 1e+00	Train: 0.9691	Val: 0.9682
lambda = 1e+01	Train: 0.9090	Val: 0.9078



Best $\lambda^* = 1e-04$ (val accuracy = 0.9698)

Using validation (not test) to select λ prevents test-set leakage.

(f)(iii) Final test accuracy at λ^*

```
In [12]: # Concatenate train + val (both already standardised with the same mu/sigma)
X_tv = np.vstack([X_train_s, X_val_s])
y_tv = np.concatenate([y_train, y_val])

w_final, b_final, _ = train_logistic_svm(
    X_tv, y_tv, lam=best_lam, eta=ETA, n_iter=N_ITER
)

test_acc_final = accuracy(X_test_s, y_test, w_final, b_final)
print(f'Final test accuracy ( $\lambda^* = \{best\_lam:.0e\}$ , retrained on train+val): {te
```

Final test accuracy ($\lambda^* = 1e-04$, retrained on train+val): 0.9601

Part (g): Summary Table and Discussion

```
In [13]: # Compute test accuracies for all models
test_acc_hinge = accuracy(X_test_s, y_test, w_hinge, b_hinge)
test_acc_log = accuracy(X_test_s, y_test, w_log, b_log)
test_acc_hinge_n = accuracy(X_test_s, y_test, w_hinge_n, b_hinge_n)
test_acc_log_n = accuracy(X_test_s, y_test, w_log_n, b_log_n)

# Training accuracies on noisy training set
tr_acc_hinge_n = accuracy(X_train_s, y_train_noisy, w_hinge_n, b_hinge_n)
tr_acc_log_n = accuracy(X_train_s, y_train_noisy, w_log_n, b_log_n)

summary = {
    'Model': [
        'Hinge SVM (clean,  $\lambda=1e-3$ )',
        'Logistic SVM (clean,  $\lambda=1e-3$ )',
        'Hinge SVM (5% label noise)',
        'Logistic SVM (5% label noise)',
        f'Logistic SVM ( $\lambda^*=\{best\_lam:.0e\}$ , train+val)',
    ],
    'Train Acc': [
        f'{train_acc_hinge:.4f}',
        f'{train_acc_log:.4f}',
        f'{tr_acc_hinge_n:.4f}',
        f'{tr_acc_log_n:.4f}',
        '-',
    ],
    'Val Acc': [
        f'{val_acc_hinge:.4f}',
        f'{val_acc_log:.4f}',
        f'{val_acc_hinge_n:.4f}',
        f'{val_acc_log_n:.4f}',
        f'{val_accs[best_idx]:.4f}',
    ],
}
```

```

    'Test Acc': [
        f'{test_acc_hinge:.4f}',
        f'{test_acc_log:.4f}',
        f'{test_acc_hinge_n:.4f}',
        f'{test_acc_log_n:.4f}',
        f'{test_acc_final:.4f}',
    ],
    '||w||2': [
        f'{np.linalg.norm(w_hinge):.4f}',
        f'{np.linalg.norm(w_log):.4f}',
        f'{np.linalg.norm(w_hinge_n):.4f}',
        f'{np.linalg.norm(w_log_n):.4f}',
        f'{np.linalg.norm(w_final):.4f}',
    ],
}

summary_df = pd.DataFrame(summary)
display(summary_df)

```

	Model	Train Acc	Val Acc	Test Acc	w ₂
0	Hinge SVM (clean, $\lambda=1e-3$)	0.9700	0.9682	0.9698	0.4488
1	Logistic SVM (clean, $\lambda=1e-3$)	0.9644	0.9698	0.9623	0.4533
2	Hinge SVM (5% label noise)	0.9186	0.9666	0.9677	0.3922
3	Logistic SVM (5% label noise)	0.9223	0.9698	0.9655	0.3831
4	Logistic SVM ($\lambda^*=1e-04$, train+val)	—	0.9698	0.9601	0.4575

Discussion

Which loss to ship to the observatory?

The logistic-loss SVM. It is smooth everywhere (no kink at $m=1$), converges more reliably with gradient descent, and — crucially — never completely ignores a correctly classified example, giving it a softer and more stable gradient signal. In an observatory context where the label catalogue may contain mislabelled RFI spikes, the bounded gradient of logistic loss ($\sigma(-m)\|x\| \leq \|x\|$) also provides intrinsic robustness to a small fraction of corrupt labels.

Did the empirical results confirm the theory?

Yes for Part (d): the scatter plot shows the three regimes of Problem 1(3) exactly. Logistic gradient norms follow the sigmoidal $\sigma(-m)\|x\|$ curve — decaying exponentially for $m \gg 0$, peaking at $\approx \frac{1}{2}\|x\|$ near $m = 0$, and saturating at $\|x\|$ for $m \ll 0$. The hinge step-function at $m = 1$ is clearly visible as a vertical drop. For Part (e), both losses show modest degradation under 5% label noise, consistent with the theoretical prediction that both have gradient magnitude capped at $\|x\|$ for extreme misclassifications.

One thing to try next:

Replace full-batch gradient descent with **mini-batch SGD** (batch size ~ 256). On a dataset of $\sim 10\,000$ examples, full-batch updates recompute gradients over all examples at every step,

which is wasteful — mini-batches provide noisy-but-cheap gradient estimates that allow many more effective parameter updates per second and often converge to a better generalising solution. A second natural extension is to **kernelize** the model with an RBF kernel (as in Problem 4) to capture non-linear structure in the pulsar features.